HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

# Nhập môn An toàn thông tin

## Chương 7. **Điều khiển truy cập**

# Nội dung

- **Tổng quan**
- Ma trận điều khiển truy cập
- Mô hình điều khiển truy cập DAC
- Mô hình điều khiển truy cập MAC
- Mô hình điều khiển truy cập RBAC
- Kiểm soát truy cập trong hệ điều hành Unix

# Điều khiển truy cập (AC) là gì?

- Trích dẫn từ Ross Anderson (Security Engineering)
  - *Its function is to control which principals (persons, processes, machines,…) have access to which resources in the system -- which files they can read, which programs they can execute, and how they share data with other principals, and so on*.

# Sự phổ biến của AC

- Lớp ứng dụng
  - Hầu khắp các ứng dụng liên quan đến doanh nghiệp
- Lớp trung gian
  - Hệ cơ sở dữ liệu
- Hệ điều hành
  - Điều khiển truy cập đến tệp tin và cổng
- Phần cứng
  - Bảo vệ bộ nhớ, các mức đặc quyền

# Nội dung

- Tổng quan
- **Ma trận điều khiển truy cập**
- Mô hình điều khiển truy cập DAC
- Mô hình điều khiển truy cập MAC
- Mô hình điều khiển truy cập RBAC
- Kiểm soát truy cập trong hệ điều hành Unix

# Ma trận điều khiển truy cập – Mô hình tổng quát bảo vệ hệ thống

- Lampson'1971
  - "Protection"
- Làm mịn bởi Graham và Denning'1972
  - "Protection---Principles and Practice"
- Harrison, Ruzzo, và Ullman'1976
  - "Protection in Operating Systems"

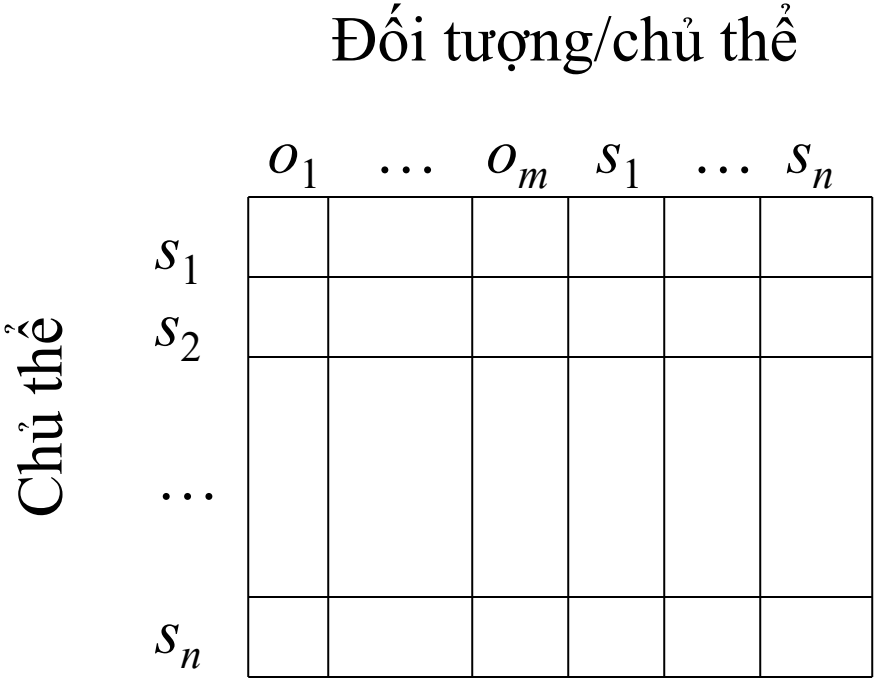
# Tổng quan

- Trạng thái bảo vệ của hệ thống
  - Mô tả bằng các thuộc tính chế độ cài đặt của hệ thống có liên quan đến bảo vệ
- Ma trận điều khiển truy cập
  - Mô tả chính xác trạng thái bảo vệ
  - Ma trận mô tả quyền của các chủ thể
  - Phần tử ma trận trong ô dữ liệu thay đổi khi có chuyển đổi trạng thái.

# Ma trận điều khiển truy cập

- S: Tập hợp các chủ thể S
- O: Tập hợp các đối tượng
- R: Tập hợp các quyền
- Ma trận điều khiển truy cập
  - Mỗi chủ thể ứng với một dòng
  - Mỗi đối tượng/chủ thể ứng với một cột
  - Mỗi ô liệt kê quyền mà chủ thể có thể sử dụng đối với đối tượng/chủ thể

# Minh họa




- Tập chủ thể $S = \{ s_1,\ldots,s_n \}$
- Tập đối tượng $O = \{ o_1,\ldots,o_m \}$
- Tập quyền $R = \{ r_1,\ldots,r_k \}$
- Ô $A[s_i, o_j] \subseteq R$
- $A[s_i, o_j] = \{r_x, \ldots, r_y\}$ có nghĩa chủ thể $s_i$ có các quyền $r_x, \ldots, r_y$ đối với đối tượng $o_j$

# Ví dụ 1

- Tiến trình *p*, *q*
- Tệp tin *f*, *g*
- Các quyền *r*, *w*, *x*, *a*, *o*

| | *f* | *g* | *p* | *q* |
|---|---|---|---|---|
| *p* | *rwo* | *r* | *rwxo* | *w* |
| *q* | *a* | *ro* | *r* | *rwxo* |

# Ví dụ 2

- Hàm thủ tục *inc_ctr*, *dec_ctr*, *manage*
- Biến *counter*
- Các quyền +, –, *call*

| | *counter* | *inc_ctr* | *dec_ctr* | *manage* |
|---|---|---|---|---|
| *inc_ctr* | + | | | |
| *dec_ctr* | - | | | |
| *manage* | | *call* | *call* | *call* |

# Cài đặt

- Lưu trữ ma trận điều khiển truy cập
  - Phân rã theo dòng: danh sách khả năng
  - Phân rã theo cột: danh sách điều khiển truy cập (ACL)
  - Biểu diễn gián tiếp:
    - Ví dụ: danh sách key và lock
    - Ví dụ: nhóm, vai trò, nhiều mức gián tiếp biểu diễn gián tiếp và nhiều khóa
- Việc thực hiện biểu diễn gián tiếp một cách chính xác và tiện lợi chính là chìa khóa để quản lý điều khiển truy cập.

# Cài đặt

**Danh sách điều khiển truy cập** (cột)

**File 1**
Joe:Read
Joe:Write
Joe:Own

**File 2**
Joe:Read
Sam:Read
Sam:Write
Sam:Own

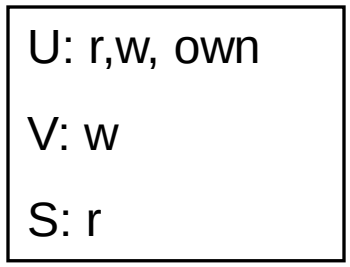
**Danh sách năng lực** (dòng)

Joe: File 1/Read, File 1/Write, File 1/Own, File 2/Read
Sam: File 2/Read, File 2/Write, File 2/Own

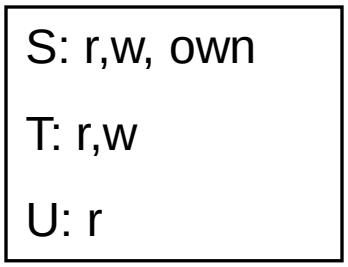
**Access Control Triples**

| Subject | Access | Object |
|---|---|---|
| Joe | Read | File 1 |
| Joe | Write | File 1 |
| Joe | Own | File 1 |
| Joe | Read | File 2 |
| Sam | Read | File 2 |
| Sam | Write | File 2 |
| Sam | Own | File 2 |

# Danh sách điều khiển truy cập




Object F




Object G

- ACL là danh sách các quyền gắn với một đối tượng
  - Ai được phép cập nhật ACL?
  - Những loại sửa đổi cập nhật nào là được phép?
  - Giải quyết thế nào nếu những đặc quyền truy nhập có mâu thuẫn?
  - Giải quyết ra sao cho thủ tục rút phép?

# Chủ nhân và Nhóm

- Ai được phép cập nhật lên ACL?
  - Một cách là giới thiệu chủ nhân của đối tượng
- Với ACL, chúng ta có thể định nghĩa bất kỳ tổ hợp quyền truy nhập nào, nhưng điều này cũng tạo ra nhiều khó khăn đối với việc quản trị
  - Khái niệm nhóm giúp cho tác vụ quản trị ACL được đơn giản hóa hơn, mang tính hàng loạt.
- Chủ nhân và nhóm có thể thay đổi

# Danh sách năng lực

- Một cách để phân hoạch ma trận theo dòng.
  - Tất cả quyền truy nhập tài nguyên của người dùng được lưu trong một cấu trúc dữ liệu, gọi là danh sách năng lực
    - Liệt kê tất cả các quyền truy nhập và năng lực mà người dùng có.
    - Ví dụ: Fred → /dev/console(RW) → fred/prog.c(RW) → fred/letter(RW) → /usr/ucb/vi(X) Jane → /dev/console(RW) → fred/prog.c(R) → fred/letter() → /usr/ucb/vi(X)

# Danh sách năng lực

- Tất cả truy nhập đến đối tượng được thực hiện thông qua năng lực truy nhập
  - Mỗi chương trình có một tập các năng lực truy nhập
  - Mỗi chương trình giữ một số lượng nhỏ năng lực truy nhập
  - Cách duy nhất một chương trình có thể có năng lực truy nhập là cấp quyền dựa trên kết quả của giao tiếp
  - Danh sách truy nhập cần được tạo ra nhỏ nhất có thể, vừa đủ để làm việc theo đúng chức năng của chủ thể (*principle of least privilege)*
- Ví dụ: Hệ điều hành EROS
  - http://www.eros-os.org/eros.html

# Nội dung

- Tổng quan
- Ma trận điều khiển truy cập
- **Mô hình điều khiển truy cập DAC**
- Mô hình điều khiển truy cập MAC
- Mô hình điều khiển truy cập RBAC
- Kiểm soát truy cập trong hệ điều hành Unix

# Mô hình Harrison-Ruzzo-Ullman

- **Điều khiển truy cập tùy nghi (DAC)**
  - Quyền truy nhập (right) cho từng cặp (chủ thể, đối tượng) có thể xác định riêng rẽ và có thể quyết định bởi chủ nhân của đối tượng (khác với **Mandatory Access Control**, được quyết định bởi chính sách hệ thống)
- Hoạt động của **HRU**
  - Hình thức hóa ma trận điều khiển truy nhập, hướng đến mô tả các hoạt động của hệ điều hành
  - Cung cấp một mô hình, đủ mạnh để mã hóa một vài cách thức điều khiển truy nhập, và đủ chặt chẽ để có thể phân tích các thuộc tính về an toàn bảo mật
  - Giới thiệu bài toán An toàn “safety problem”
  - Chứng minh rằng bài toán an toàn chỉ có thể giải được khi đưa về những trường hợp đặc biệt với một số điều kiện đủ tốt

# Các thao tác cơ bản

- **create subject** *s*; **create object** *o*
  - Tạo mới dòng, cột trong ACM; tạo mới một cột trong ACM
- **destroy subject** *s*; **destroy object** *o*
  - Xóa dòng, cột của ACM; xóa cột của ACM
- **enter** *r* **into** *A*[*s*, *o*]
  - Thêm vào quyền *r* cho chủ thể *s* đối với đối tượng *o*
- **delete** *r* **from** *A*[*s*, *o*]
  - Thu hồi quyền *r* khỏi chủ thể *s* đối với đối tượng *o*

# Tạo tệp dữ liệu

- Tiến trình *p* tạo tệp dữ liệu mới *f* với các quyền *r* và *w*

```
command create_file(p, f)
    create object f;
    enter own into A[p, f];
    enter r into A[p, f];
    enter w into A[p, f];
end
```

# Mono-Operational Commands

- Cấp quyền làm chủ cho một tiến trình *p* đối với tệp *g*

```
command make_owner(p, g)
    enter own into A[p, g];
end
```

- Mono-operational command
  - Trong lệnh này chỉ có một thao tác đơn nguyên thủy

# Conditional Commands

- Tiến trình *p* cấp cho *q* quyền *r* đối với tệp *f*, nếu *p* là chủ sở hữu của *f*

```
command grant_read_file_1(p, f, q)
    if own in A[p, f]
    then
        enter r into A[q, f];
end
```

- Mono-conditional command
  - Trong lệnh này chỉ có một điều kiện đơn

# Điều khiển truy nhập tùy nghi (DAC)

- Không có định nghĩa chặt chẽ
- Sử dụng phổ biến trong các hệ điều hành hiện đại
- Gắn liền với sự sử dụng khai niệm chủ nhân của mỗi đối tượng
- Chủ nhân kiểm soát khả năng truy nhập của các chủ thể khác đối với đối tượng
- Cho phép cấp phát quyền truy cập đến các đối tượng khác

# Hạn chế của DAC

- DAC không thể ngăn chặn
  - Trojan horse (Ngựa thành Troy)
  - Mã độc
  - Lỗi phần mềm
  - Người sử dụng nội bộ có ý đồ xấu
- Không thể kiểm soát được luồng thông tin

# Nội dung

- Tổng quan
- Ma trận điều khiển truy cập
- Mô hình điều khiển truy cập DAC
- **Mô hình điều khiển truy cập MAC**
- Mô hình điều khiển truy cập RBAC
- Kiểm soát truy cập trong hệ điều hành Unix

# Điều khiển truy nhập cưỡng chế

- *Đối tượng:* Phân loại mức bảo mật

  Ví dụ: grades=(confidential, {student-info})
- *Chủ thể:* Nhãn bảo mật

  Ví dụ: Joe=(confidential, {student-info})
- *Luật truy nhập:* định nghĩa bằng cách so sánh mức độ mật của đối tượng yêu cầu với nhãn bảo mật của chủ thể

  Ví dụ: Chủ thể có thể đọc đối tượng chỉ khi nhãn(chủ thể) cao hơn nhãn(đối tượng)

# Điều khiển truy nhập cưỡng chế

- Cấp quyền khi thỏa mãn luật kiểm soát truy nhập

  Ví dụ, Joe muốn đọc dữ liệu grades.

  label(Joe)=(confidential,{student-info})

  label(grades)=(confidential,{student-info})

  Joe được cấp quyền để đọc dữ liệu grades

- *Granularity* of access rights!

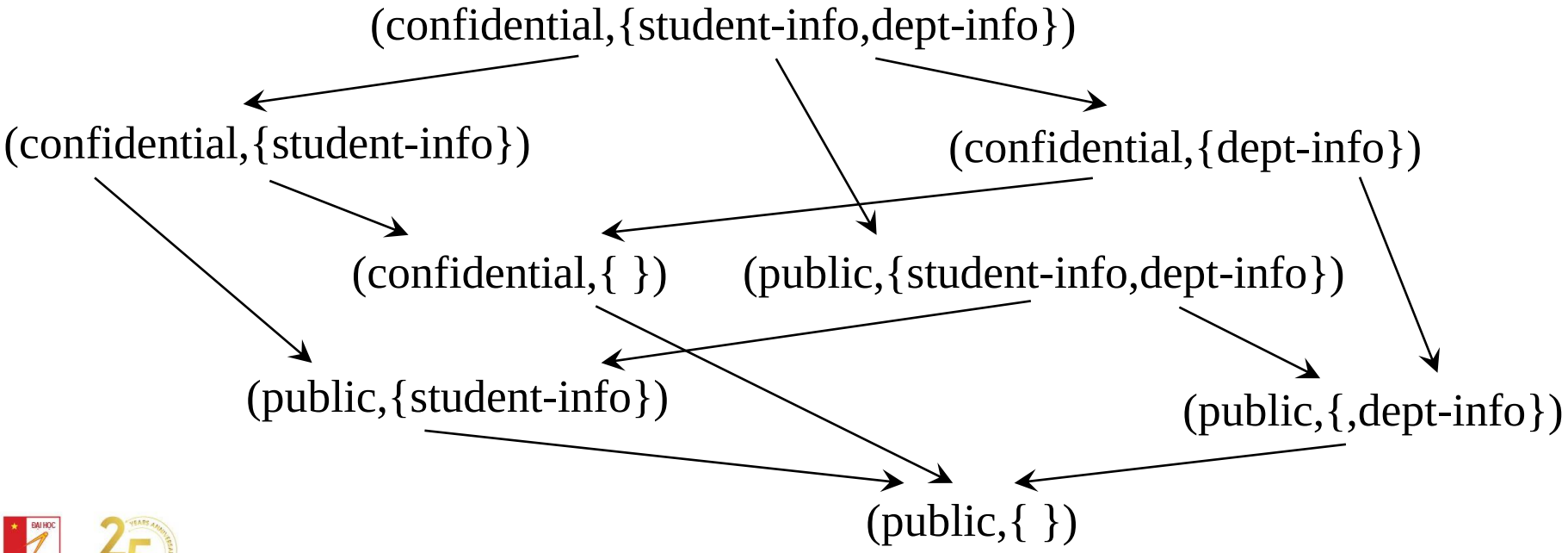
# Điều khiển truy nhập cưỡng chế

***Cấp bậc của chủ thể (nhãn)***: (A,C)

A – cấp/mức

C – nhóm thể loại

Ví dụ: A = confidential > public , C = {student-info, dept-info}



# Điều khiển truy nhập cưỡng chế

- *Ưu hơn* ($\geq$): nhãn l=(A,C) là ưu thế hơn nhãn l'=(A',C') nếu và chỉ nếu $A \geq A'$ and $C \supseteq C'$

  Ví dụ: (confidential,{student-info}) ≥ (public,{student-info})

  NHƯNG KHÔNG PHẢI

  (confidential, {student-info}) ≥ (public,{student-info, department-info})

# Mô hình Bell- LaPadula (BLP)

- Bảo vệ tính mật
- Lattice-based access control
  - Chủ thể
  - Đối tượng
  - Nhãn bảo mật
- Hỗ trợ quản trị phi tập trung

# BLP Reference Monitor

- Tất cả yêu cầu truy nhập đều phải đi qua bộ phận kiểm soát monitor
- Không thể vượt mặt
- Chấp nhận yêu cầu truy nhập nếu và chỉ nếu trang thái hệ thống đầu ra thỏa mãn tất cả các thuộc tính bảo mật
- *Chủ thể tin cậy: Chủ thể được tin tưởng không xâm hại đến hệ thống*

# Luật cơ bản của BLP

- *Bảo mật đơn giản*: một chủ thể *s* sẽ chỉ được phép thực hiện thao tác đọc đối với đối tượng *o* nếu nhãn của *s* là ưu thế hơn nhãn của *o*
  - No read up
  - *Áp dụng cho tất cả chủ thể*
  - Chủ thể *s* được đọc đối tượng *o* nếu và chỉ nếu $L(o) \leq L(s)$ và *s* được cấp phép đọc đối với *o*
    - **Chú ý**: Kết hợp điều khiển truy nhập cưỡng chế (quan hệ của mức độ bảo mật) và điều khiển truy nhập tùy nghi (yêu cầu cấp phép)

# Luật cơ bản của BLP

*-*property*: một chủ thể *s* chỉ được thực hiệ thao tác viết lên đối tượng *o khi* nhãn của *o* là ưu thế hơn nhãn của *s*

- No write down
- Chỉ áp dụng với đối tượng không được coi là tin cậy
  - Chủ thể *s* có thể viết lên đối tượng *o* nếu và chỉ nếu $L(s) \leq L(o)$ và *s* được cấp phép để viết lên *o*
  - Chú ý: kết hợp điều khiển truy nhập cưỡng chế (quan hệ của mức độ bảo mật) và điều khiển truy nhập tùy nghi (yêu cầu cấp phép)

# Ví dụ

| *security level* | *subject* | *object* |
|---|---|---|
| Top Secret | Tamara | Personnel Files |
| Secret | Samuel | E-Mail Files |
| Confidential | Claire | Activity Logs |
| Unclassified | Ulaley | Telephone Lists |

- Tamara có thể đọc tất cả tệp dữ liệu
- Claire không thể đọc tệp dữ liệu Personnel hoặc E-Mail
- Ulaley có thể đọc dữ liệu Telephone Lists

# Levels and Lattices

- Mức độ bảo mật: (*nhãn*, *nhóm thể loại*)
  - ( Top Secret, { NUC, EUR, ASI } )
  - ( Confidential, { EUR, ASI } )
  - ( Secret, { NUC, ASI } )
- (*A*, *C*) *ưu thế hơn* (*A′*, *C′*) nếu và chỉ nếu $A' \leq A$ and $C' \subseteq C$
  - (Top Secret, {NUC, ASI}) *ưu thế hơn* (Secret, {NUC})
  - (Secret, {NUC, EUR}) *ưu thế hơn* (Confidential,{NUC, EUR})
  - (Top Secret, {NUC}) *ưu thế hơn* (Confidential, {EUR})

# Nội dung

- Tổng quan
- Ma trận điều khiển truy cập
- Mô hình điều khiển truy cập DAC
- Mô hình điều khiển truy cập MAC
- **Mô hình điều khiển truy cập RBAC**
- Kiểm soát truy cập trong hệ điều hành Unix

# Ý nghĩa của RBAC

- Các hệ thống đa người dùng
- Các hệ thống đa ứng dụng
- Gắn quyền với vai trò
- Sự gắn quyền với vai trò thường là lâu dài
- Gần với trực giác: Năng lực, nghĩa vụ và trách nhiệm

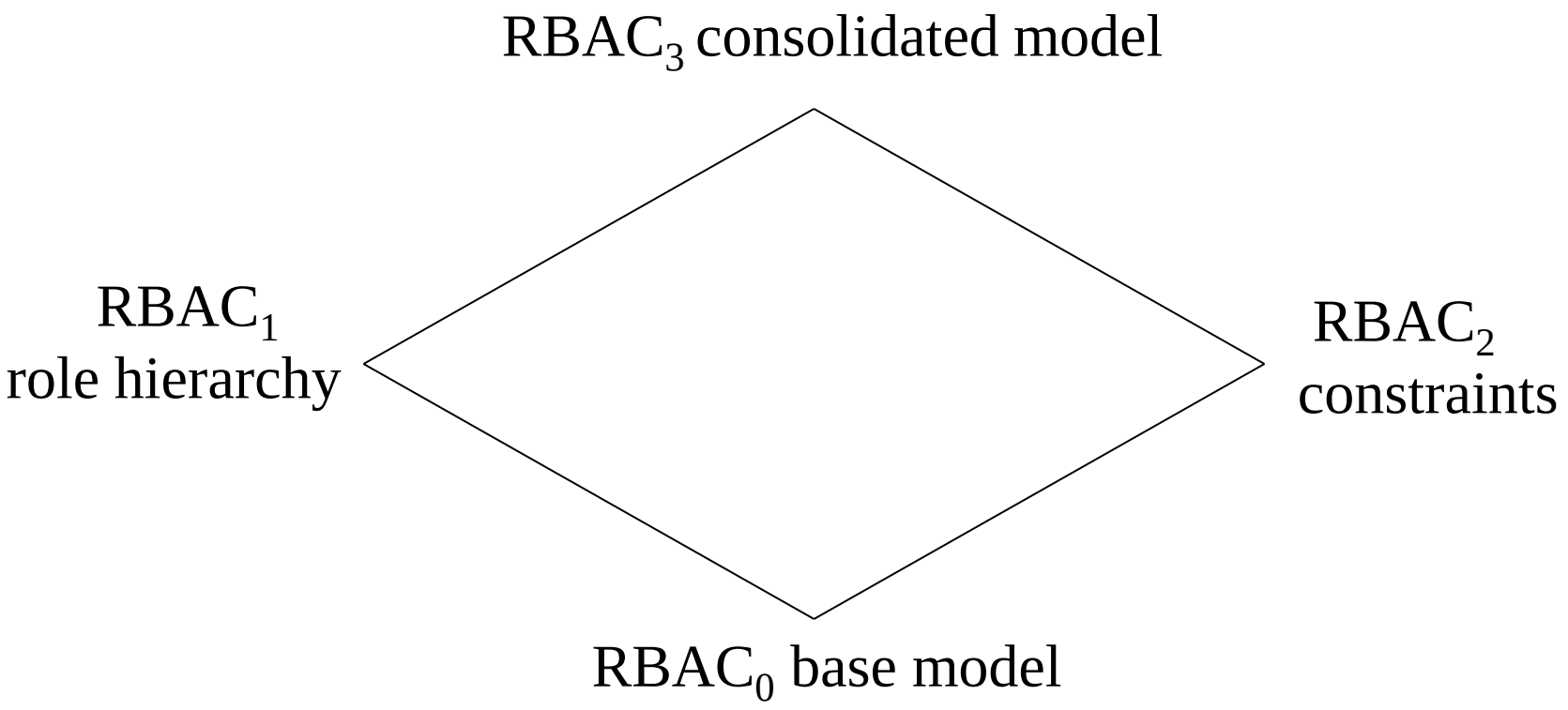
# Ý nghĩa của RBAC

- Phù hợp với mô hình doanh nghiệp
  - Chia sẻ trách nhiệm
  - Ủy quyền
- Linh hoạt: Dễ dàng thay đổi để phù hợp với các yêu cầu bảo mật mới
- Hỗ trợ
  - Đặc quyền tối thiểu
  - Chia sẻ trách nhiệm
  - Khái quát hóa thông tin dữ liệu

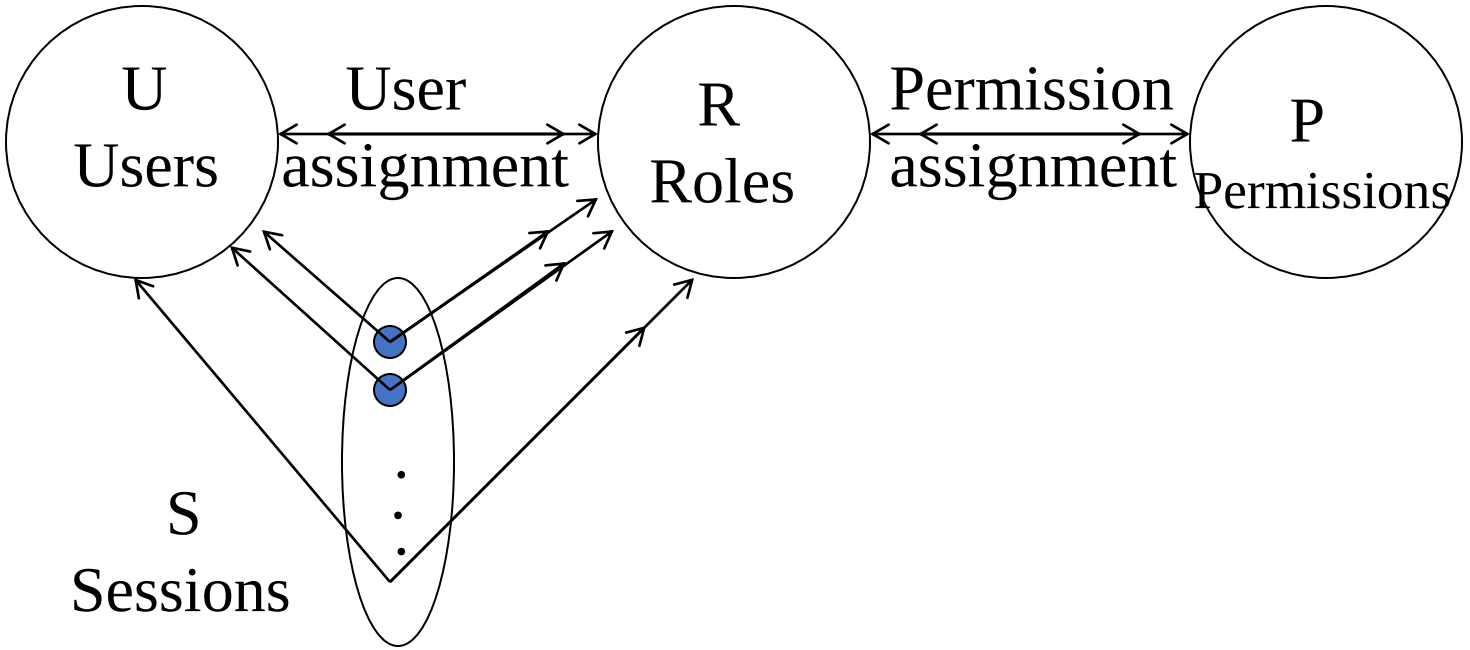
# Vai trò

- **Nhóm người dùng**: Tập người dùng, mỗi người dùng có thể có các quyền khác nhau
- **Vai trò**: Khái niệm trung gian giữa tập các người dùng và một tập các quyền khai thác
- **RBAC** độc lập với DAC và MAC (chúng có thể cùng tồn tại)
- RBAC là trung tính với chính sách (**policy neutral)**: Cấu hình RBAC sẽ xác định chính sách muốn áp đặt vào hệ thống

# RBAC


RBAC3 consolidated model
RBAC1
role hierarchy
RBAC2
constraints
RBAC0 base model

# $RBAC_0$


U
Users
User
assignment
R
Roles
Permission
assignment
P
Permissions
S
Sessions

# $RBAC_0$

- Người dùng
- Vai trò: chức năng nhiệm vụ (vị trí công việc)
- Quyền truy nhập khai thác: phê duyệt chế độ truy nhập
  - Always positive
  - Abstract representation
  - Có thể áp dụng một hoặc nhiều đối tượng

# $RBAC_0$

- UA: user assignments
  - Quan hệ nhiều – nhiều
- PA: Permission assignment
  - Quan hệ nhiều – nhiều
- Session: quan hệ một - nhiều giữa người dùng và các vai trò
  - Nhiều vai trò có thể kích hoạt đồng thời
  - Quyền khai thác: Tập hợp các quyền khai thác từ nhiều vai trò
  - Mỗi phiên làm việc chỉ tương ứng với một người dùng
  - Người dùng có thể có nhiều phiên làm việc tại một thời điểm

# Đối tượng dữ liệu của $RBAC_0$

- **U**sers, **R**oles, **P**ermissions, **S**essions
- PA $\subseteq$ P x R (quan hệ nhiều nhiều)
- UA $\subseteq$ U x R (quan hệ nhiều nhiều)
- user: S → U, hàm ánh xạ phiên $s_i$ đến một người dùng user($s_i$)
- roles: S → $2^R$, hàm ánh xạ phiên $s_i$ đến một tập các vai trò:
  - roles($s_i$) $\subseteq$ {r | (user($s_i$),r) $\in$ UA} và $s_i$ được cấp quyền $U_{r \in roles(s_i)}$ {p | (p,r) $\in$ PA}

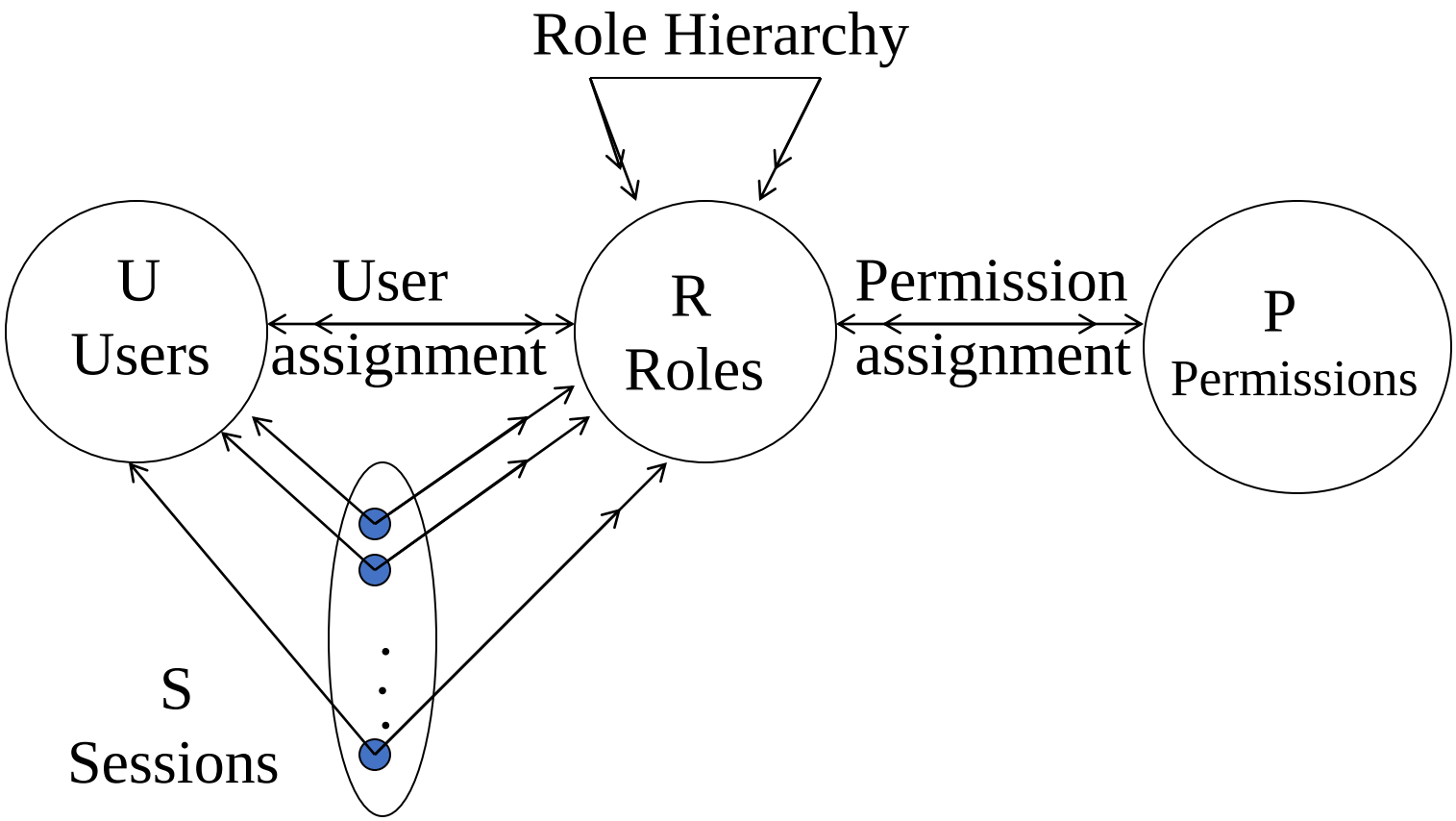
# $RBAC_0$

- Quyền khai thác chỉ được áp dụng cho các đối tượng dữ liệu tài nguyên
- Quyền khai thác KHÔNG áp dụng cho chính các đối tượng dữ liệu RBAC
- Quyền quản trị: thực hiện sửa đổi U,R,S,P
- Phiên: người dùng điều khiển để
  - Kích hoạt tập con các vai trò được cấp phép
  - Thay đổi vai trò trong một phiên

# $RBAC_1$

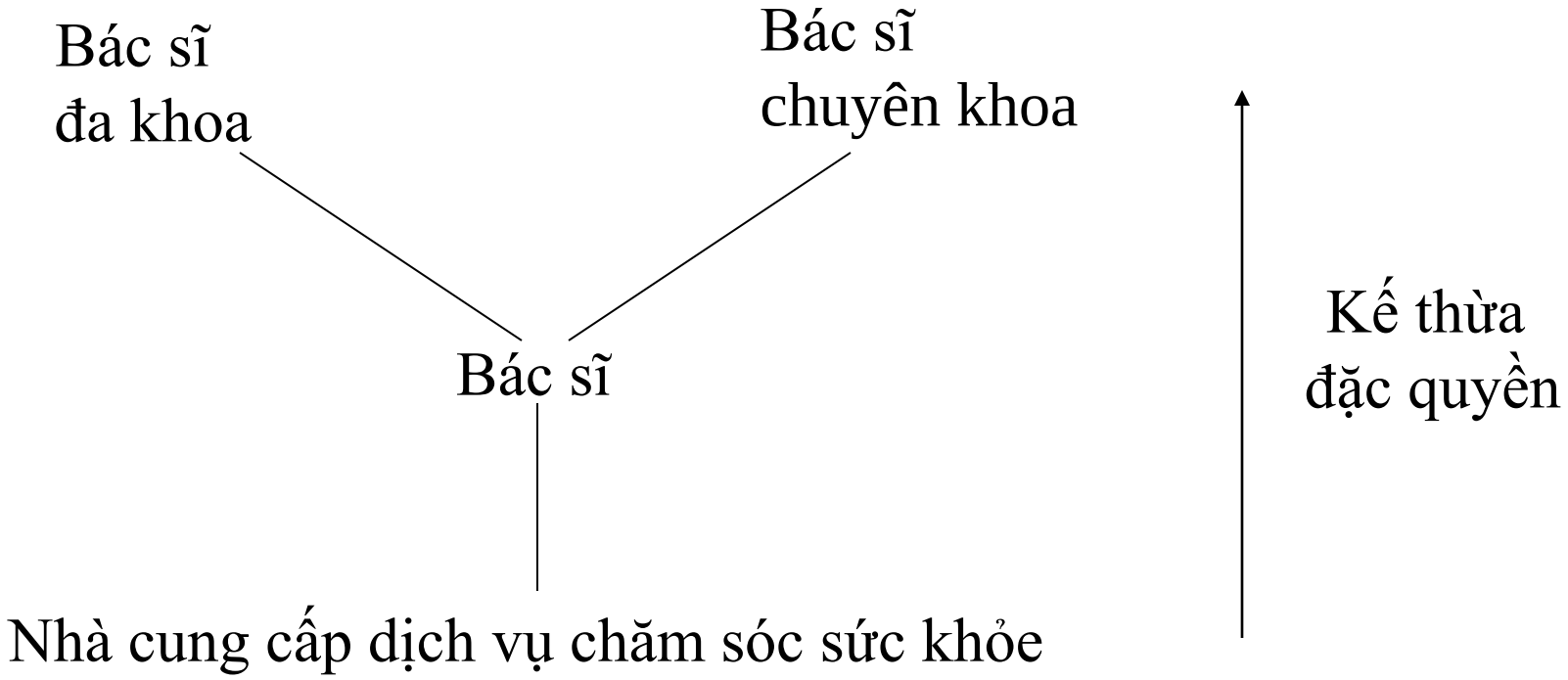
- Tổ chức phân cấp các vai trò
- Kế thừa quyền sử dụng các quyền truy nhập của các vai trò cấp dưới
- Quan hệ bán thứ tự
  - Phản xạ
  - Truyền ứng
  - Bắc cầu

# $RBAC_1$


Role Hierarchy
U
Users
User
assignment
R
Roles
Permission
assignment
P
Permissions
S
Sessions

# $RBAC_1$

Phân cấp vai trò



# Đối tượng dữ liệu của $RBAC_1$

- Tương tự như $RBAC_0$: **U**sers, **R**oles, **P**ermissions, **S**essions, $PA \subseteq P \times R$, $UA \subseteq U \times R$, user: $S \rightarrow U$, hàm ánh xạ phiên $s_i$ đến một người dùng $user(s_i)$
- $RH \subseteq R \times R$, quan hệ bán thứ tự ($\geq$ ưu thế hơn)
- roles: $S \rightarrow 2^R$, hàm ánh xạ phiên $s_i$ đến một tập các vai trò
  - $roles(s_i) \subseteq \{r \mid (\exists r' \geq r)\ [(user(s_i), r') \in UA]\}$ và $s_i$ được cấp quyền $\bigcup_{r \in roles(s_i)} \{p \mid (\exists r'' \leq r)\ [(p, r'') \in PA]\}$

# Nội dung

- Tổng quan
- Ma trận điều khiển truy cập
- Mô hình điều khiển truy cập DAC
- Mô hình điều khiển truy cập MAC
- Mô hình điều khiển truy cập RBAC
- **Kiểm soát truy cập trong hệ điều hành Unix**

# Nguyên tắc chung

- Người dùng, nhóm người dùng, các tiến trình, các tệp dữ liệu
  - Mỗi người dùng có một UID duy nhất
  - Mỗi nhóm người dùng có một GID duy nhất
  - Mỗi tiến trình có một PID duy nhất
  - Người dùng có thể thuộc nhiều nhóm GID
  - Đối tượng điều khiển truy nhập: Tệp dữ liệu, Thư mục
- Tổ chức các đối tượng
  - Tệp dữ liệu được tổ chức theo cấu trúc phân cấp
  - Tệp dữ liệu có thể thuộc nhiều thư mục
  - Thư mục cũng là một dạng tệp đặc biệt
  - Trong UNIX, quyền truy nhập vào thư mục không có tính kế thừa

# Bit quyền truy nhập của tệp dữ liệu

- Quyền truy nhập:
  - Read:  điều khiển việc đọc nội dung của tệp dữ liệu
  - Write: điều khiển việc thay đổi nội dung của tệp dữ liệu
  - Execute: điều khiển việc thực thi tệp
- Nhiều tác vụ chỉ được thực thi bởi chủ sở hữu của tệp tin
- Vị trí lưu trữ các bit mô tả quyền truy nhập?
  - Mỗi tệp tin/thư mục tương ứng với một i-node.
  - Dạng tệp tin, quyền truy nhập, chủ sở hữu UID và nhóm chủ sở hữu GID được lưu trên địa tại inode của tệp tin hoặc thư mục

# Bit quyền truy nhập của thư mục

- Read: hiển thị tên các tệp dữ liệu trong thư mục
- Execution: cho phép duyệt thư mục
  - Tìm kiếm, cho phép tìm inode # từ tên tệp
  - ‘chdir’ chuyển đến thư mục để thực thi
- Write + execution: tạo mới hoặc xóa các tệp tin trong một thư mục
  - Không yêu cầu quyền truy nhập đối với tệp tin
- Truy nhập đường dẫn một tệp tin: cần quyền thực thi đối với tất cả thư mục trong đường dẫn

# 3 Bit quyền truy nhập

- **Ví dụ quyền truy nhập**

  drwxr-xr-x

  - Bit đầu tiên: đối tượng có phải thư mục?
  - Ba vị trí tiếp theo: Quyền mà người chủ có thể thực hiện
    - Nếu người dùng là chủ sở hữu của tệp tin, các quyền có thể áp dụng gồm r/w/x
  - Ba vị trí tiếp theo: Quyền của nhóm chủ sở hữu
    - Nếu người dùng thuộc một nhóm chủ, các quyền có thể áp dụng gồm r/w/x
  - Ba vị trí tiếp theo: Quyền của người sử dụng khác
    - Áp dụng khi người dùng không phải chủ sở hữu hoặc nhóm chủ
- **Vị trí lưu trữ các bit quyền truy nhập?**
  - Mỗi tệp tin/thư mục tương ứng với một i-node.
  - Dạng tệp tin, quyền truy nhập, chủ sở hữu UID và nhóm chủ sở hữu GID được lưu trên địa tại inode của tệp tin hoặc thư mục

# Người dùng vs. Chủ thể

- Permission bits cho biết khả năng có thể truy nhập vào một tệp tin
  - ➔ Chủ thể của hành động truy nhập là các tiến trình
  - Khi chủ thể truy nhập vào tệp tin, hệ thống kiểm tra để biết người sử dụng mà tiến trình đại diện và lấy quyền
- Vấn đề: nếu như công việc nào đó đòi hỏi quyền truy nhập lớn hơn quyền thực có của chủ thể?
  - Chương trình **passwd** đòi hỏi cập nhật tệp tin lưu trữ mật khẩu hệ thống, người dùng không thể thay đổi nếu không có đặc quyền quản trị hệ thống
  - Cần nhớ rằng, chương trình ***passwd*** được tạo ra để người dùng thường sử dụng

# UID chủ vs. UID hiệu lực

- Mỗi tiến trình có 3 UIDs
  - UID chủ (ruid): Người chủ của tiến trình
  - UID hiệu lực (euid): Được dùng trong hầu hết các quyết định về điều khiển truy nhập, tương tự như ruid trừ trường hợp ngoại lệ
  - ID lưu trạng thái trước (suid): Lưu trạng thái trước của euid nếu có sự thay đổi
- Ba nhóm chủ GIDs
  - GID chủ
  - GID hiệu lực
  - GID lưu trạng thái trước

# Cờ setuid

- Sử dụng cho một tệp tin để
  - Cấp phép cho một vài tiến trình cụ thể có đặc quyền cao hơn so với đặc quyền cơ bản của người dùng thường
  - Khi thiết lập, uid hiệu lực của tiến trình bị gọi sẽ được chuyển thành UID của người chủ của tệp

# Nguyên tắc hoạt động

- Khi một tiến trình được tạo thông qua lệnh *fork*
  - Nó kế thừa cả ba UIDs từ tiến trình mẹ
- Khi một tiến trình thực hiện một tệp tin bằng lệnh exec

  Nếu tệp tin không cài đặt cờ setuid: Nó kế thừa 3 UID

  Trường hợp khác // cờ setuid đặt

  euid của tiến trình = ruid của tệp tin

  suid = euid lưu trạng thái trước
- Giải quyết vấn đề passwd và các vấn đề tương tự?
  - Passwd là sở hữu của người quản trị (root) và được đặt cờ setuid
  - Khi một tiến trình gọi thực hiện nó, UID hiệu lực sẽ được đặt là root trong thời gian tiến trình passwd hoạt động
- Điểm yếu về ann toàn cho hệ điều hành Unix nếu như cơ chế tạm mượn đặc quyền này bị lợi dụng